

Module 1: Optimization

Module 2: Convolutional Networks

Module 3: Graph + Sequence Models

Module 4: Attention + Transformers

Attention (Lecture 18)

- **Seq2Seq bottleneck.** Encoder compresses whole input into single state h_T ; decoder state and outputs:

$$h_t = f(x_t, h_{t-1}), \quad s_t = f(y_{t-1}, s_{t-1}), \quad \hat{y}_t = g(s_t).$$

Fixed-size h_T is a bottleneck.

- **Teacher forcing.** During training, feed true previous token y_{t-1}^{true} instead of sampled $\hat{y}_{t-1}$.

- **Q/K/V projections.**

$$q_t = W_q s_t, \quad k_i = W_k h_i, \quad v_i = W_v h_i.$$

- **Dot-product similarity.**

$$\text{score}(q_t, k_i) = q_t^\top k_i.$$

- **Scaled dot-product attention.**

$$\alpha_{t,i} = \frac{\exp(q_t^\top k_i / \sqrt{d})}{\sum_j \exp(q_t^\top k_j / \sqrt{d})}, \quad c_t = \sum_i \alpha_{t,i} v_i.$$

- **Matrix form.**

$$Q = XW_q, \quad K = XW_k, \quad V = XW_v,$$

$$\text{Attn}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right)V.$$

- **Self-attention.** Q, K, V from same sequence (e.g., tokens of a sentence) so each token attends to all others.

- **Cross-attention.** Queries Q from decoder; keys/values K, V from encoder states $\{h_i\}$.

- **Multi-head attention.**

$$H_j = \text{softmax}\left(\frac{QW_q^{(j)}(KW_k^{(j)})^\top}{\sqrt{d_h}}\right)VW_v^{(j)},$$

$$\text{MHA}(Q, K, V) = \text{Concat}(H_1, \dots, H_h)W_o.$$

- **Multi-query attention (MQA).** All heads share K, V ; only queries use per-head projections. Reduces memory and speeds decoding.

- **Grouped MQA.** Heads partitioned into groups; each group shares its own K, V . Interpolates between full MHA and full MQA.

Module 5: Parameter-Efficient Finetuning

Module 6: Advanced ML Paradigms